

YouTube Shorts Recommendation System

Resume Bullets & Interview Guide

RESUME BULLET POINTS WITH QUANTIFIED IMPACT

1. Built real-time recommendation pipeline for YouTube Shorts using two-tower neural architecture with FAISS-based candidate generation and Transformer ranking model, increasing user engagement by 21% and achieving <100ms p95 latency serving 50M+ daily active users across mobile and web clients
2. Developed multi-objective ranking model balancing watch time, likes, and creator diversity using Transformer-based encoders with attention mechanisms, processing 500M+ daily impressions and improving content discovery metrics by 18% while maintaining 94% user satisfaction score
3. Engineered session-based personalization system integrating implicit feedback signals (rewatch rate, skip patterns, scroll velocity) using RNNs and Graph Neural Networks, resulting in 27% increase in session duration and 15% improvement in next-day retention rates
4. Designed candidate generation system combining collaborative filtering (matrix factorization with 128-dim embeddings), trending algorithms, and graph-based similarity scoring via ScaNN, retrieving top-1000 candidates with >85% Recall@1000 in <50ms for 2B+ video catalog
5. Deployed production ML pipeline using TensorFlow Extended (TFX) and Google Cloud Dataflow with automated feature engineering, model training, and validation, reducing training time from 12 hours to 2.5 hours and enabling daily model updates with 99.97% serving availability
6. Implemented comprehensive cold-start strategy using demographic clustering, content-based filtering on video metadata embeddings, and trending boosting algorithm, improving new user engagement by 34% and achieving 78% accuracy on cold-start recommendations within first 5 interactions
7. Built automated A/B testing framework with statistical significance testing (t-tests, bootstrap confidence intervals) and performance monitoring using Kubeflow Pipelines and Vertex AI, reducing experimentation cycle from 2 weeks to 3 days and enabling 40+ concurrent experiments
8. Optimized model serving architecture using TensorFlow Serving with GPU acceleration and Redis caching for user/video embeddings, achieving sub-100ms end-to-end latency at p95, reducing infrastructure costs by 32% through efficient batching, and handling 15K QPS during peak traffic

DEEP DIVE INTERVIEW QUESTIONS & ANSWERS

1. Explain the architecture of your YouTube Shorts recommendation system. How does the two-stage retrieval and ranking pipeline work?

The system uses a two-stage funnel: Stage 1—Candidate Generation retrieves top-1000 videos from 2B+ catalog using FAISS approximate nearest neighbor search on 128-dim user/video embeddings trained via matrix factorization. This stage prioritizes recall (>85% Recall@1000) and runs in <50ms. Stage 2—Ranking Model is a Transformer-based neural network that scores and reranks the 1000 candidates using 200+ features (user history, video metadata, engagement signals, temporal features, session context). The ranking model optimizes for multiple objectives: watch time (primary), likes, shares, and creator diversity. Final top-20 recommendations are returned to the client. This architecture balances computational efficiency (can't score 2B videos in real-time) with recommendation quality (detailed ranking on smaller candidate set).

2. How did you design the multi-objective ranking model? What trade-offs did you consider?

The ranking model uses weighted multi-task learning with three heads: watch time prediction (regression), engagement prediction (likes/shares, binary classification), and diversity scoring. Loss function: $L = 0.6 * L_{\text{watch}} + 0.25 * L_{\text{engagement}} + 0.15 * L_{\text{diversity}}$. Weights tuned via grid search and A/B testing. Trade-offs: Watch time alone causes filter bubbles (same creator repeatedly). Adding diversity term reduced watch time by 3% but increased creator discovery by 40% and improved long-term retention by 8%. Engagement signals balance viral content vs quality. Used focal loss for engagement to handle class imbalance. Architecture: Transformer encoder (6 layers, 8 attention heads) processes sequence of last 50 user interactions, cross-attention with candidate video features, final MLP predicts scores. Trained on 30 days of interaction data (500M samples), batch size 2048, 100K steps.

3. Walk me through your feature engineering. What features were most important for Shorts recommendations?

Features split into four categories: (1) User features—watch history embeddings (last 50 videos, averaged), engagement rate (likes/shares per view), session features (time on app, scroll velocity), demographics (age, location), temporal patterns (active hours, day of week). (2) Video features—content embeddings (from video understanding model), creator ID, category, duration, upload recency, popularity metrics (views, likes, watch time). (3) Interaction features—user-creator affinity (past views of creator), category preferences, co-watch patterns. (4) Context features—time of day, device type, network speed. Most important features via SHAP analysis: user watch history embeddings (35% importance), user-creator affinity (18%), video popularity decay (12%), session scroll velocity (9%). Implemented temporal decay for older interactions (exponential decay with 7-day half-life). All features normalized and embedded before feeding to Transformer.

4. How did you handle the cold start problem for new users and new videos?

New users: (1) Demographic-based recommendations—cluster users by age/location, serve popular content in that cohort. (2) Onboarding flow—ask users to select interests (3-5 categories), immediately retrieve top trending videos in those categories. (3) Rapid learning—after first 3 interactions, switch to lightweight personalized model using only recent engagement. (4) Exploration bonus—add 20% random trending videos to maintain diversity. New videos: (1) Content-based filtering—extract video embeddings using pre-trained video understanding model (analyzing frames, audio, text), find similar videos by cosine similarity. (2) Creator history—if creator has previous videos, leverage creator embeddings. (3) Trending boost—new videos get 2x score multiplier for first 24 hours to gather initial engagement data. (4) Small-scale testing—serve to 1% of users first, if engagement >threshold, promote to wider audience. Achieved 78% cold-start recommendation accuracy (vs 92% for warm users).

5. Explain your candidate generation approach using FAISS. Why FAISS over other ANN methods?

Candidate generation uses learned embeddings: user embedding (128-dim) from aggregating user's watch history through neural network, and video embeddings (128-dim) learned jointly via two-tower architecture. At serving time, retrieve top-1000 nearest video embeddings to user embedding using FAISS IndexIVFPQ (Inverted File with Product Quantization). Why FAISS: (1) Speed—50ms for k-NN search over 2B vectors on GPU. (2) Memory efficiency—Product Quantization compresses 128-dim float32 (512 bytes) to 64 bytes (8x compression). (3) Accuracy—tuned nprobe parameter balancing recall vs latency (nprobe=32 gives 85% Recall@1000). (4) GPU support—10x faster than CPU. Alternatives considered: ScaNN (Google, competitive but FAISS better GPU support), Annoy (Spotify, slower), HNSW (memory intensive). Index rebuilt daily with new embeddings, old index cached for rollback. Quantization loses <2% recall vs brute-force but 50x faster.

6. How did you incorporate implicit feedback signals like scroll velocity and skip patterns?

Implicit signals capture user intent better than explicit likes. Implemented: (1) Scroll velocity—fast scrolling (>3 videos/sec) indicates disinterest, slow scrolling indicates engagement. Binned into 3 categories (slow/medium/fast), one-hot encoded. (2) Rewatch rate—if user scrolls back to video, strong positive signal. Added binary feature 'was_rewatched'. (3) Skip patterns—skipping within 2 seconds indicates dislike. Time-to-skip encoded as feature, combined with exponential decay. (4) Dwell time—even without like, if user watches >80% of video, it's positive signal. Created 'completion_rate' feature. (5) Screenshot/share without like—indicates value. Used RNN (GRU) to model sequence of these implicit actions within session, capturing patterns like 'user skips comedy but watches dance'. GNN captures relationships between videos user engages with, creating interaction graph. These features improved NDCG@10 by 12% vs explicit signals alone.

7. Describe your deployment pipeline using TFX and Google Cloud Dataflow. How did you achieve daily model updates?

TFX pipeline orchestrates end-to-end ML workflow: (1) ExampleGen—ingests yesterday's interaction logs from BigQuery. (2) StatisticsGen + SchemaGen—validates data quality, detects schema drift. (3) Transform—feature engineering (scaling, encoding, embeddings lookup), outputs transformed dataset. (4) Trainer—trains model on Cloud TPUs (takes 2.5 hours for 30-day data window). (5) Evaluator—validates model on holdout set, checks NDCG@10 >baseline. (6) Pusher—deploys to Vertex AI if validation passes, with gradual rollout (canary 5% → 25% → 100% over 6 hours). Cloud Dataflow processes 500M daily events in parallel (scales to 1000 workers). Key optimizations: incremental training (warm-start from yesterday's model saves 60% training time), feature caching (Redis stores user/video embeddings), parallel evaluation (splits validation across 10 workers). Automated rollback if live metrics (CTR, watch time) degrade >5%. Monitoring with Vertex AI Model Monitoring for drift detection. Entire pipeline runs daily at 2am, new model live by 8am.

8. How did you design and execute A/B tests? What metrics did you track and how did you ensure statistical significance?

A/B testing framework: (1) User assignment—hash user_id to assign 50% control (baseline model), 50% treatment (new model). Assignment sticky for duration of test. (2) Metrics tracked—Primary: watch time per session, session duration. Secondary: CTR, likes, shares, next-day retention, creator diversity (entropy). (3) Sample size—power analysis for 2% MDE (Minimum Detectable Effect) with 80% power and 5% significance requires 15K users per group, ran for 7 days (total 210K users). (4) Statistical tests—t-test for continuous metrics (watch time), chi-squared for binary

(CTR). Bootstrap confidence intervals (10K resamples) for robustness. (5) Multiple testing correction—Bonferroni correction for 6 metrics (adjusted alpha = $0.05/6 = 0.0083$). Results: new model increased watch time by 4.2% ($p < 0.001$), CTR by 2.8% ($p = 0.002$), no significant change in retention. Shipped to production. Kubeflow Pipelines automates: metric collection, significance testing, automated reports in Grafana.

9. What were the main challenges in achieving <100ms p95 latency? How did you optimize the serving infrastructure?

Latency breakdown: candidate generation (50ms), feature fetching (30ms), ranking inference (15ms), network (5ms) = 100ms total. Optimizations: (1) FAISS on GPU—reduced search from 200ms (CPU) to 50ms. Index loaded in GPU memory. (2) Feature caching—Redis cluster caches user embeddings (7-day TTL) and video embeddings (never expire). Cache hit rate 92%. Reduced feature fetching from 150ms to 30ms. (3) Model optimization—TensorFlow Serving with GPU batching. Dynamic batching (max batch size 64, max wait 5ms) improves throughput 8x. Model quantization (FP16) speeds up inference 1.7x with <0.5% accuracy loss. (4) Pre-computation—candidate pool refreshed every 5 minutes (not per request). Trending videos pre-ranked. (5) Request coalescing—batch multiple concurrent requests for same user. (6) Monitoring—p95/p99 latency dashboards in Grafana, alerts if p95 >120ms. Geographic distribution (serving in 12 regions) reduces network latency. Result: p95 latency 95ms (within SLA), p99 140ms, handles 15K QPS peak traffic.

10. How did you balance creator diversity with user engagement? What was your approach to avoiding filter bubbles?

Filter bubbles occur when users only see content from same creators. Implemented: (1) Diversity regularization in loss function— $L_{\text{diversity}} = -H(\text{creator_distribution})$ where H is entropy. Higher entropy = more diverse creators. Weight = 0.15 in multi-objective loss. (2) Hard constraints—max 2 videos from same creator in top-20 recommendations. (3) Creator exposure tracking—maintain per-user counter of impressions per creator, down-rank over-exposed creators. (4) Exploration—20% of recommendations are randomly sampled from trending or category-diverse set. (5) MMR (Maximal Marginal Relevance) reranking—iteratively select videos that maximize relevance to user while minimizing similarity to already-selected videos. Diversity weight $\lambda = 0.3$. Trade-offs: Pure relevance (no diversity) gave 3% higher immediate watch time but 8% lower 7-day retention and 12% lower creator satisfaction (measured via creator survey). With diversity optimization: 15% of users discover 3+ new creators weekly. Creator Gini coefficient (inequality measure) decreased from 0.82 to 0.67. Balanced approach maximizes long-term platform health.

11. Walk me through your offline evaluation metrics. How do you ensure offline metrics correlate with online performance?

Offline metrics: (1) Ranking metrics—NDCG@10 (primary, measures ranking quality with position discounting), MRR (Mean Reciprocal Rank), Precision@K, Recall@K. (2) Retrieval metrics—Recall@1000 for candidate generation (measures if true positives in top-1000). (3) Diversity metrics—Creator entropy, category coverage, Intra-List Diversity (avg pairwise dissimilarity in top-20). (4) Prediction metrics—AUC for engagement prediction, RMSE for watch time prediction. Holdout validation: 80/10/10 split (train/validation/test), temporal split (last 3 days for test). Offline-online correlation: tracked correlation between offline NDCG and online watch time across 15 A/B tests—correlation coefficient 0.78. Challenges: (1) Position bias—users more likely to engage with top items regardless of relevance. Corrected with Inverse Propensity Scoring. (2) Delayed feedback—likes/shares happen later. Used 48-hour attribution window. (3) Selection bias—only observe engagement on served items. Used counterfactual evaluation with logged data. Validation: any model with NDCG@10 <0.40 or >5% drop vs baseline auto-rejected.

12. How did you handle real-time personalization using session features? What's the architecture for low-latency feature updates?

Real-time personalization captures within-session behavior: (1) Session features—videos watched this session (last 10), scroll velocity, time on app, skip rate this session. Updated every 30 seconds. (2) Architecture—Client sends session context with each request → API Gateway → Feature Service (FastAPI) → fetches static features from Redis → computes dynamic session features → combines → sends to Ranking Model → returns recommendations. (3) Stateful session tracking—session ID generated on app open, stored in Redis (30-min TTL). Session state includes current video queue, engagement actions, timestamp. (4) Incremental updates—after each interaction, client sends event to Kafka → Flink processes in real-time → updates session state in Redis within 500ms. (5) Session modeling—RNN (GRU with 3 layers, 128 hidden units) encodes session sequence, output fed to ranking model. Captures temporal patterns like "user getting bored" (decreasing watch time) → recommend different content category. Personalization improved session-level metrics: 27% longer session duration, 19% more videos watched per session vs non-personalized baseline.

13. Explain your approach to monitoring model performance in production. What metrics do you track and how do you detect degradation?

Multi-layer monitoring: (1) Business metrics—Watch time per user (primary KPI), CTR, session duration, DAU, retention rates. Tracked hourly with 7-day moving average baseline. Alert if >5% drop for >2 hours. (2) Model metrics—prediction distribution drift (KS test on predicted watch time), feature distribution drift (compare to training distribution), inference latency (p50/p95/p99). (3) Data quality—missing feature rate, schema violations, input distribution shifts. (4) Infrastructure—QPS, error rate, GPU utilization, cache hit rate. Dashboards in Grafana with 50+ charts. Drift detection: (1) Statistical tests—KL divergence between current and reference distributions. Alert if KL >0.1. (2) PSI (Population Stability Index) for feature drift. (3) Online vs offline metric gap—compare online CTR to model's predicted CTR, alert if gap >10%. Automated responses: (1) Canary rollback—if metrics degrade during gradual rollout, auto-rollback. (2) Feature fallback—if feature fetch fails, use default value. (3) Model fallback—keep 2 previous model versions, can instant-switch if needed. On-call alerts via PagerDuty. Weekly model health reviews analyzing trends.

14. How did you optimize model training? What techniques did you use to reduce training time from 12 hours to 2.5 hours?

Training optimizations: (1) Data pipeline—TFRecords format with compression reduces I/O time by 60%. Parallel data loading (16 workers) and prefetching (buffer size 1000). (2) Hardware—Cloud TPU v3 pods (256 cores) vs single GPU (100x faster). Mixed precision training (FP16) speeds up 1.8x. (3) Distributed training—data parallelism across 32 workers with synchronous SGD. All-reduce for gradient aggregation. (4) Efficient architecture—reduced Transformer from 12 to 6 layers (2% NDCG drop but 3x faster). Replaced some attention with FFN layers (linear layers faster than attention). (5) Incremental learning—warm-start from yesterday's model, train only on last 7 days of new data (vs full 30-day retraining). Saves 60% time with no accuracy loss. (6) Smart sampling—oversample rare categories, undersample over-represented creators (reduces dataset by 40% with better balance). (7) Early stopping—monitor validation NDCG every 1K steps, stop if no improvement for 5K steps. (8) Gradient checkpointing—trades computation for memory, allows 2x batch size. From 12 hours → 2.5 hours enabled daily updates, fresher recommendations, faster iteration.

15. Looking back at this project, what were the biggest challenges and what would you do differently? What were your key learnings?

Biggest challenges: (1) Cold start—required separate strategy and careful tuning. Built dedicated cold-start model. (2) Balancing objectives—watch time vs diversity vs creator satisfaction. Multi-objective loss tuning took 6 weeks of A/B testing. (3) Latency constraints—<100ms at scale required extensive optimization (caching, GPU serving, model compression). (4) Data quality—20% of sessions had incomplete data. Implemented robust missing value handling. (5) Filter bubbles—initial model created echo chambers, required diversity mechanisms. What I'd do differently: (1) Start with simpler model—began with complex Transformer, should have started with MLP baseline to establish performance ceiling. (2) Earlier A/B testing—ran offline experiments for 4 weeks before first A/B test. Should have tested baseline model online earlier. (3) Better feature engineering upfront—spent weeks adding features iteratively. Should have designed comprehensive feature schema from start. (4) Invest in evaluation framework earlier—built offline evaluation metrics in Week 4, should have been Week 1. Key learnings: (1) Simple models (collaborative filtering) provide strong baseline. (2) Offline metrics (NDCG) must be validated against online metrics (watch time). (3) Diversity is crucial for long-term engagement. (4) Production ML is 20% modeling, 80% infrastructure (serving, monitoring, data pipelines). (5) Latency constraints fundamentally shape architecture decisions. Result: Successfully shipped system serving 50M+ DAU with 21% engagement lift. Project taught me end-to-end production ML at scale.